Model order reduction for a transient reaction-diffusion problem using the Laplace transform

Keyvan Attarian[*], Amine Ammar[†]

Printed May 22, 2026

Abstract

Model order reduction for parametric transient partial differential equations (PDEs) typically suffers from the high computational cost of predicting long time horizons. To alleviate this, we propose a surrogate modeling strategy that leverages the Laplace transform to map the time domain into a highly compressed set of complex frequency fields. We systematically compare two non-linear compression approaches using autoencoders: Spatial Laplace Autoencoders (SLAE), which compress individual frequency frames, and Latent Laplace Autoencoders (LLAE), which apply the transform directly within a low-dimensional latent space. Our results demonstrate that end-to-end training of the surrogate models, combined with a lightweight UNet residual corrector to mitigate spectral truncation artifacts, significantly improves prediction accuracy. Ultimately, the AE-based methods bridge the gap between fast surrogate predictions and the autoencoder reconstruction floor, establishing a highly efficient and accurate framework for transient PDE emulation.

Contents

[*]Ecole polytechnique, Institut polytechnique de Paris

[†]ENSAM

1 Introduction

High-fidelity simulations of parametric transient partial differential equations (PDEs), such as those governing combustion, reactive transport, and atmospheric dispersion, rely on computationally expensive numerical solvers. In many-query contexts like optimization, uncertainty quantification, real-time control, and digital twins, the complete spatiotemporal field $U(x, t; \boldsymbol{\mu})$ must be evaluated for thousands of parameter values $\boldsymbol{\mu}$ [4, 16]. The prohibitive cost of these repeated evaluations motivates the development of surrogate models capable of reproducing the full dynamics in milliseconds. Applications such as air quality and pollution forecasting concretely illustrate this need: global physical models are accurate but unusable in real-time without approximation [8, 2].

Classical model order reduction (MOR) methods [9], such as Reduced Basis (RB) and Proper Orthogonal Decomposition (POD) combined with interpolation, construct a low-dimensional linear subspace by projecting high-fidelity solutions onto spatial modes. These approaches are theoretically well-founded and highly effective for smooth dynamics. However, they suffer from a reconstruction floor linked to the linearity of the basis: for problems with sharp transients, localized structures, or strongly non-linear dynamics (such as convection-diffusion-reaction equations), the number of required modes explodes. The treatment of time in these methods is either sequential (time-stepping on the projected ROM) or space-time (variational), both of which impose a computational cost that grows with the temporal horizon [5, 11, 19]. The transition toward non-linear manifolds becomes necessary as linear techniques struggle [3]. Our results confirm this limitation: a linear SVD baseline in the Laplace domain reaches 100% relative error on our test case.

To overcome the linear floor, two main families of data-driven approaches have emerged alongside the advent of Physics-Informed Neural Networks [21]. On one hand, deep learning-based ROMs (DL-ROMs) [12, 3] jointly learn a non-linear manifold and reduced dynamics via convolutional autoencoders, demonstrating significant gains over linear ROMs for parametric non-linear PDEs. On the other hand, neural operators directly learn mappings between function spaces (i.e., field-to-field mappings). Notable examples include DeepONet [14] and its POD-based extensions [15], and the Fourier Neural Operator (FNO) [13], which employs convolution in the Fourier space. Because they are inherently designed to map an input function to an output function, they are not naturally suited for direct parameter-to-field predictions where the input is a low-dimensional scalar vector. Furthermore, these approaches treat time as an additional continuous dimension, imposing a high memory and computational cost for long sequences and limiting their ability to capture long-horizon transients without dense training data. Parameter-conditioned U-Nets [10] represent a lighter alternative for 2D convection-diffusion-reaction PDEs, yet they still operate directly in the time domain.

Spectral decompositions offer an alternative perspective for transient PDEs. FNO [13] parameterizes the integral kernel directly in the Fourier space, giving it remarkable efficiency on periodic or quasi-stationary problems. However, the Fourier transform implicitly assumes periodicity and is poorly suited for transient signals: it struggles to represent the natural exponential growth or decay of solutions and suffers from Gibbs artifacts in the presence of temporal discontinuities. The Laplace transform $s = \gamma + i\omega$ generalizes Fourier by adding a real decay axis γ that naturally encodes transients, exponential stability, and non-periodic regimes. Each frequency field $\hat{U}(s_k)$ is a static complex spatial field, structurally simpler to compress than an entire temporal sequence. The Laplace Neural Operator (LNO) [1] illustrates this advantage: by exploiting the pole-residual relationship in the Laplace domain, LNO surpasses FNO on transient responses and non-periodic signals for benchmark ODEs and PDEs. Regarding linear reduction, recent works exploit the Laplace domain to construct reduced bases: Reyes [22] shows via the Paley-Wiener theorem that the Laplace domain produces exponentially accurate POD bases for evolution problems, and Goeckner et al. [6] extend this to parametric PDEs via contour integral methods. Nonetheless, these approaches remain fundamentally linear. The remaining gap is to replace this linear compression with non-linear autoencoders jointly learned with a parametric surrogate, enabling end-to-end differentiable training thanks to the exact invertibility of the transform [7].

This paper proposes a surrogate modeling strategy for parametric transient PDEs that exploits the Laplace transform as a spectral compression tool for the spatiotemporal field, combined with non-linear autoencoders. Two strategies are systematically compared: compression in the spatial domain frequency-by-frequency (SLAE), and compression in the temporal latent space prior to transformation (LLAE/LSLAE). End-to-end training is made possible by the differentiability of the inversion operator. A lightweight UNet residual corrector addresses Gibbs artifacts stemming from spectral truncation. Our contributions are threefold: (1) Formulation and comparison of two AE architectures in the Laplace domain (SLAE vs LLAE/LSLAE); (2) Demonstration that end-to-end training is indispensable for bridging the gap between the reconstruction floor and the surrogate prediction; (3) Introduction of a differentiable UNet residual corrector for mitigating spectral artifacts.

The remainder of this paper is organized as follows. Section 2 describes the methodology, detailing the Laplace transform, surrogate architectures, and the end-to-end training pipeline. Section 3 presents the numerical results and performance comparisons. Finally, Section 4 provides a discussion and concludes the work. The appendices detail the governing equations and architectural hyperparameters.

2 Methodology

2.1 Laplace Transform and Inversion

The forward Laplace transform is evaluated at discrete frequencies $s = \gamma + i\omega$ using a temporal quadrature rule (e.g., trapezoidal rule with weights w_n):

$$\hat{U}(s) = \int_0^T U(t)\, e^{-st}\, \mathrm{d}t \approx \Delta t \sum_{n=0}^{N_t-1} w_n\, U(t_n)\, e^{-st_n} \tag{1}$$

Since the physical temporal field $U(t)$ is strictly real-valued, its Laplace transform satisfies the conjugate symmetry property $\hat{U}(\bar{s}) = \overline{\hat{U}(s)}$, where $\bar{s} = \gamma - i\omega$. Consequently, we restrict our numerical evaluation to frequencies with non-negative imaginary parts, $s_k = \gamma + i\omega_k$ with $\omega_k \geq 0$ for $k \in \{0, \ldots, N_l\}$. This symmetry effectively doubles the number of available frequency fields for a given computational cost, reducing the forward evaluation overhead by a factor of two while providing the full spectral information required for inversion.

To recover the temporal field $U(t)$ from this reduced frequency set, we consider two inversion methods:

Regularised Inversion (Tikhonov). For an arbitrary set of frequencies s_k, the inversion is ill-posed. Let $\boldsymbol{F} \in \mathbb{C}^{N \times N_t}$ be the forward evaluation matrix mapping the real temporal sequence to the complex frequency domain, such that $\boldsymbol{F}_{k,n} = \Delta t\, w_n\, e^{-s_k t_n}$. Let $\hat{\boldsymbol{u}} \in \mathbb{C}^N$ and $\boldsymbol{u} \in \mathbb{R}^{N_t}$ be the flattened vectors corresponding to the frequency predictions $\hat{U}(s_k)$ and the temporal field $U(t_n)$, respectively. Taking advantage of the real-valued constraint on $\boldsymbol{u}$, we formulate the reconstruction as a regularised complex least-squares problem restricted to the non-negative frequency spectrum:

$$\boldsymbol{u}^* = \arg\min_{\boldsymbol{u}\in\mathbb{R}^{N_t}} \|\boldsymbol{F}\boldsymbol{u} - \hat{\boldsymbol{u}}\|^2_{\mathbb{C}^N} + \alpha_t \|\boldsymbol{D}_t\boldsymbol{u}\|_2^2 + \lambda \|\boldsymbol{u}\|_2^2 \tag{2}$$

where $\boldsymbol{D}_t \in \mathbb{R}^{(N_t-1)\times N_t}$ is the first-difference operator penalising temporal roughness, and $\lambda > 0$ is a Tikhonov ridge regularisation parameter. The corresponding optimality condition (normal equations) yields a real symmetric linear system:

$$\left(\Re(\boldsymbol{F}^H\boldsymbol{F}) + \alpha_t \boldsymbol{D}_t^T\boldsymbol{D}_t + \lambda \boldsymbol{I}\right)\boldsymbol{u}^* = \Re(\boldsymbol{F}^H\hat{\boldsymbol{u}}) \tag{3}$$

By solving this system, the conjugate frequencies are implicitly accounted for through the real-part operator $\Re(\cdot)$, ensuring a physically consistent, real-valued temporal reconstruction. Since this Tikhonov regularised numerical inversion is implemented via a linear least-squares solve (e.g., using `torch.linalg.solve`), the inversion operator is fully differentiable. This approach is well-suited for gradient-based training and arbitrary frequency selections.

Bromwich Contour (Special Case). When the frequencies $s_k = \gamma + i\omega_k$ are uniformly spaced along a vertical line in the complex plane such that ω_k corresponds to the discrete Fourier frequencies, the forward transform can be formulated as a Discrete Fourier Transform (DFT) of the exponentially damped sequence:

$$\hat{U}(s_k) = \text{FFT}\left\{U(t_n)\Delta t\, w_n e^{-\gamma t_n}\right\}_k \tag{4}$$

The inverse transform is then efficiently and exactly computed using the Inverse Fast Fourier Transform (IFFT), without the need for regularisation:

$$U(t_n) = \frac{e^{\gamma t_n}}{\Delta t\, w_n}\Re\left(\text{IFFT}\left\{\hat{U}(s_k)\right\}_n\right) \tag{5}$$

2.2 SVD Baseline in the Laplace Domain

A linear baseline approach applies a truncated Singular Value Decomposition (SVD) independently to each Laplace frequency s_k. The complex spatial field $\hat{U}(s_k)$ is separated into its real and imaginary components, which are then projected onto pre-computed, fixed spatial bases $\boldsymbol{V}^{(k,r)}$ and $\boldsymbol{V}^{(k,i)}$:

$$\Re(\hat{U}(s_k)) \approx \sum_{j=1}^{k_{\text{svd}}} c_j^{(k,r)}\boldsymbol{V}_j^{(k,r)}, \qquad \Im(\hat{U}(s_k)) \approx \sum_{j=1}^{k_{\text{svd}}} c_j^{(k,i)}\boldsymbol{V}_j^{(k,i)} \tag{6}$$

For each frequency k, dedicated Multilayer Perceptrons (MLPs), parameterized by weights $\boldsymbol{\theta}$, act as surrogates to predict the normalised coefficients $\boldsymbol{c}^{(k,r)}$ and $\boldsymbol{c}^{(k,i)}$ directly from the physical parameters $\boldsymbol{\mu}$. Specifically, independent MLPs are employed for the real and imaginary parts, governed by the following mappings:

$$\boldsymbol{c}^{(k,r)} = f_{\boldsymbol{\theta}}^{(k,r)}(\boldsymbol{\mu}), \quad \boldsymbol{c}^{(k,i)} = f_{\boldsymbol{\theta}}^{(k,i)}(\boldsymbol{\mu}) \tag{7}$$

where f_θ denotes the parametrized neural network function. These MLPs are trained to minimize the mean squared error (MSE) between the predicted and true SVD projection coefficients:

$$\mathcal{L}_{\text{SVD}}^{(k)} = \left\| f_{\boldsymbol{\theta}}^{(k,r)}(\boldsymbol{\mu}) - \boldsymbol{c}^{(k,r)} \right\|_2^2 + \left\| f_{\boldsymbol{\theta}}^{(k,i)}(\boldsymbol{\mu}) - \boldsymbol{c}^{(k,i)} \right\|_2^2 \tag{8}$$

The time series $U(t)$ is subsequently reconstructed by multiplying the predicted coefficients with their respective bases, combining them into a complex field, and applying the inverse Laplace transform. However, this approach is fundamentally constrained by the irreducible truncation error of the linear spatial bases, particularly at higher frequencies where the spatial modes exhibit complex, highly localized structures.

2.3 Autoencoders in the Laplace Domain

To design non-linear reduced-order models, we investigate and compare two alternative autoencoder (AE) strategies:

- **Spatial Laplace AE (SLAE):** The Laplace transform is computed in the physical domain, and a frequency-conditioned autoencoder compresses and reconstructs each complex frequency frame independently.
- **Latent Laplace AE (LLAE):** A time-conditioned autoencoder first compresses the physical time-series frames into a low-dimensional latent space, and the Laplace transform is subsequently applied to the temporal latent codes.

In both configurations, we incorporate an L_2 "ridge" regularization penalty on the latent representations. This constraint is crucial as it enforces a sufficiently regular and smooth latent space structure, which subsequently makes it much easier for the surrogate model to learn the mapping from the physical parameters $\boldsymbol{\mu}$ to the latent codes z (as detailed in Section 2.4).

Both architectures share common conditioning components: a sinusoidal coordinate encoding and Feature-wise Linear Modulation (FiLM). Below, we describe these mechanisms in a general manner using a coordinate $\tau \in [0,1]$ that represents either the frequency ratio $f_k = k/(N_{\text{half}} - 1)$ in the spatial approach, or the normalised time step t/T in the latent approach.

Sinusoidal coordinate encoding. The coordinate $\tau \in [0,1]$ is embedded via sinusoidal basis functions inspired by NeRF positional encoding [17]:

$$\boldsymbol{e}(\tau) = \text{MLP}\Big(\big[\sin(2^i \pi\tau),\ \cos(2^i \pi\tau)\big]_{i=0}^{L-1}\Big), \qquad L = 8. \tag{9}$$

A plain scalar τ would conflate nearby coordinates and fail to capture sharp transitions in either the frequency or time domains. The multi-scale sinusoidal basis explicitly represents τ at L different resolutions, providing the network with a rich, non-saturating signal that allows fine discrimination across the entire range.

FiLM conditioning. To share encoder/decoder weights across all coordinates (frequencies or time steps) while adapting to each specific configuration, the FiLM mechanism [20] is employed. At each convolutional layer ℓ, the coordinate embedding $\boldsymbol{e}(\tau)$ is projected to a channel-wise gain $\alpha_\ell \in \mathbb{R}^{C_\ell}$ and bias $\delta_\ell \in \mathbb{R}^{C_\ell}$, modulating the activations element-wise:

$$\boldsymbol{x}_\ell \;\leftarrow\; \big(\boldsymbol{x}_\ell \odot (1 + \alpha_\ell(\boldsymbol{e}(\tau)))\big) \;+\; \delta_\ell(\boldsymbol{e}(\tau)). \tag{10}$$

The residual formulation $(1 + \alpha_\ell)$ initialises the modulation close to the identity, which stabilises early training. All coordinates share the same convolutional weights, and the FiLM parameters act as a lightweight per-coordinate adapter at each layer, enabling a single autoencoder to process the full spectrum or temporal sequence without training separate model copies.

2.3.1 Spatial Laplace AE (SLAE)

In SLAE, the Laplace transform operates directly on the time-domain spatial pixels, transforming the sequence of 2D grids $U(t)$ into K complex-valued frequency frames $\hat{U}(s_k)$. A frequency-conditioned encoder (utilizing 3 strided convolutional downsampling blocks with LeakyReLU activations and FiLM) compresses each 2-channel (Real and Imaginary) frame independently into a real latent vector $z(s_k) \in \mathbb{R}^{d_z}$:

$$U(t) \xrightarrow{\mathcal{L}_{\text{pixel-wise}}} \hat{U}(s_k) \xrightarrow{\text{Encoder}(f_k)} z(s_k) \xrightarrow{\text{Decoder}(f_k)} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t) \tag{11}$$

The corresponding decoder upsamples $z(s_k)$ via transposed convolutions. To prevent cross-talk between the real and imaginary components during the final projection, the decoder bifurcates into two separate refinement blocks (each consisting of successive convolutions with kernel sizes of 3 and 1), outputting the complex channels independently. The spatial autoencoder is trained by minimizing a loss function comprising the spatial reconstruction error in the Laplace domain and L_2 regularization on the latent representations:

$$\mathcal{L}_{\text{SLAE}} = \left\|\tilde{U}(s_k) - \hat{U}(s_k)\right\|^2_{\mathbb{C}^N} + \beta \left\|z(s_k)\right\|^2_2 \tag{12}$$

where $\|\cdot\|^2_{\mathbb{C}^N}$ represents the combined squared L_2 norm of the real and imaginary parts of the spatial grid, and β is a regularisation hyperparameter.

2.3.2 Latent Laplace AE (LLAE)

Alternatively, LLAE applies the transform within the low-dimensional latent space. The single-channel physical fields $U(t)$ are first encoded frame-by-frame by a similar convolutional encoder into latent codes $z(t) \in \mathbb{R}^{d_z}$, but conditioned on the time ratio t/T. The forward Laplace transform is then applied along the time dimension of the latent sequence:

$$U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\mathcal{L}} \hat{z}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t) \tag{13}$$

This formulation fundamentally reduces the computational overhead of the transform and regularises the dynamics by enforcing temporal consistency within the latent space. However, it introduces a significant computational drawback: LLAE requires passing all N_t physical time frames through the autoencoder (both encoding and decoding) during training and inference, whereas SLAE only processes the N_L frequency frames. Since $N_t \gg N_L$, the LLAE approach is computationally much heavier. Consequently, under a fixed computational budget, we are constrained in terms of the maximum capacity (or volume) of the autoencoder that can be deployed. Specifically, the loss function is defined as:

$$
\begin{aligned}
\mathcal{L}_{\text{LLAE}} = & \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|U_{\text{rec}}(t_n) - U(t_n)\|_2^2 \\
& + \beta_{\text{latent}} \frac{1}{N_t} \sum_{n=0}^{N_t-1} \|\tilde{z}(t_n) - z(t_n)\|_2^2 \\
& + \beta \frac{1}{N_l + 1} \sum_{k=0}^{N_l} \|\hat{z}(s_k)\|_{\mathbb{C}^{d_z}}^2
\end{aligned} \tag{14}
$$

which combines the physical-time spatial reconstruction error, the latent reconstruction error between the target latent codes $z(t_n)$ and the reconstructed codes $\tilde{z}(t_n)$, and an L_2 regularization penalty on the complex Laplace-domain latent representation $\hat{z}(s_k)$.

Latent SVD Laplace AE (LSLAE) Variant. To simplify the learning task of the surrogate model and reduce the latent dimensionality, we introduce a variant called Latent SVD Laplace AE (LSLAE), conceptually related to rank reduction autoencoders [18]. In this model, we perform a truncated Singular Value Decomposition (SVD) on the time-domain latent codes $z(t) \in \mathbb{R}^{d_z}$ offline across the training set, yielding a projection matrix $V \in \mathbb{R}^{d_z \times d_{\text{SVD}}}$ (with $d_{\text{SVD}} < d_z$). The temporal latents are projected onto this basis as $G(t) = z(t)V \in \mathbb{R}^{d_{\text{SVD}}}$, and the forward Laplace transform is subsequently applied to $G(t)$ to obtain the lower-dimensional complex frequencies $\hat{G}(s_k) \in \mathbb{C}^{d_{\text{SVD}}}$. The complete autoencoding pipeline is represented as:

$$
U(t) \xrightarrow{\text{Encoder}(t)} z(t) \xrightarrow{\text{SVD } V} G(t) \xrightarrow{\mathcal{L}} \hat{G}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{G}(t) \xrightarrow{\text{SVD } V^T} \tilde{z}(t) \xrightarrow{\text{Decoder}(t)} U_{\text{rec}}(t) \tag{15}
$$

2.4 Surrogate Training

The surrogate model replaces the encoding steps by predicting the latent representation directly from $\boldsymbol{\mu}$. Architecturally, the surrogate model $g_{\boldsymbol{\theta}}$ maps the physical parameters $\boldsymbol{\mu}$ to the predicted latent representations in the Laplace domain:

$$
z_{\text{pred}}(s_k) = g_{\boldsymbol{\theta}}^{(k)}(\boldsymbol{\mu}), \quad \hat{z}_{\text{pred}}(s_k) = g_{\boldsymbol{\theta}}^{(k)}(\boldsymbol{\mu}), \quad \text{or} \quad \hat{G}_{\text{pred}}(s_k) = g_{\boldsymbol{\theta}}^{(k)}(\boldsymbol{\mu}) \tag{16}
$$

where $g_{\boldsymbol{\theta}}^{(k)}$ represents the k-th frequency-specific head of the surrogate network, predicting the complex latents for SLAE, LLAE, and LSLAE respectively. Specifically, the forward pass of the surrogate-based reconstruction pipeline is defined for SLAE as:

$$
\boldsymbol{\mu} \xrightarrow{g_{\boldsymbol{\theta}}} z_{\text{pred}}(s_k) \xrightarrow{\text{Decoder}} \tilde{U}(s_k) \xrightarrow{\mathcal{L}^{-1}} U_{\text{rec}}(t) \tag{17}
$$

for LLAE as:

$$
\boldsymbol{\mu} \xrightarrow{g_{\boldsymbol{\theta}}} \hat{z}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t) \tag{18}
$$

and for LSLAE as:

$$
\boldsymbol{\mu} \xrightarrow{g_{\boldsymbol{\theta}}} \hat{G}_{\text{pred}}(s_k) \xrightarrow{\mathcal{L}^{-1}} \tilde{G}_{\text{pred}}(t) \xrightarrow{V^T} \tilde{z}_{\text{pred}}(t) \xrightarrow{\text{Decoder}} U_{\text{rec}}(t) \tag{19}
$$

The entire pipeline is trained end-to-end to minimise the total physical-time loss:

$$\mathcal{L}_{\text{total}} = \alpha_{\text{lat}}||w_{\text{pred}} - w_{\text{true}}||^2 + \alpha_{\text{spat}}||U_{\text{rec}} - U||^2 \tag{20}$$

where the latent terms w_{pred} and w_{true} represent $z_{\text{pred}}(s_k)$ and $z_{\text{true}}(s_k)$ for SLAE, $\hat{z}_{\text{pred}}(s_k)$ and $\hat{z}_{\text{true}}(s_k)$ for LLAE, or $\hat{G}_{\text{pred}}(s_k)$ and $\hat{G}_{\text{true}}(s_k)$ (where $\hat{G}_{\text{true}}(s_k) = \mathcal{L}\{z_{\text{true}}(t)V\}_k$) for LSLAE. This end-to-end backpropagation enables gradients from the spatiotemporal MSE loss to flow backward through both the non-linear spatial decoder and the temporal inverse transform, optimally distributing representation capacity across frequencies and updating the surrogate weights, decoder filters, and the projection basis V (for LSLAE).

3 Results

3.1 Problem Setting

We consider the emulation of the methane mass fraction Y in a two-dimensional parametric transient reacting flow. The transport equation is formulated as a convection-diffusion-reaction problem over a spatial domain $\Omega \subset \mathbb{R}^2$ and a time interval $t \in [0, T]$:

$$\frac{\partial(\rho Y)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y) - \nabla \cdot (\mu_{\text{eff}} \nabla Y) = \dot{\omega}_{\text{eff}}(Y; \boldsymbol{\mu}) \tag{21}$$

where ρ is the fluid density, $\mathbf{U}$ is the velocity field, μ_{eff} is the effective diffusivity, and $\dot{\omega}_{\text{eff}}$ is the effective reaction rate. A detailed formulation of the complete system of governing equations is provided in Appendix A. The system is governed by a vector of three physical parameters $\boldsymbol{\mu} = [k, \theta, C]^\top \in \mathbb{R}^3$, specifically representing:

- k: Diffusion parameter,
- θ: Injection angle,
- C: Injection rate (kg/m^3/s).

The spatial domain is discretized into a uniform grid of $N_x = N \times N = 128 \times 128$ points, and the temporal domain is resolved into $N_t = 150$ discrete time steps with a uniform step size $\Delta t = 1\,\text{s}$. Consequently, the discretized methane mass fraction field Y is represented as a two-dimensional tensor (or matrix) of shape $N_x \times N_t$. The dataset consists of high-fidelity numerical simulations evaluated across different parameter configurations sampled from the physical parameter space. We utilize a dataset partitioned into ?? training simulations and ?? test samples for evaluating the surrogate models.

The performance of the models is evaluated quantitatively using the relative L^2 error over the held-out test set, defined as:

$$\epsilon_{\text{rel}} = \frac{||Y_{\text{true}} - Y_{\text{pred}}||_2}{||Y_{\text{true}}||_2} \tag{22}$$

We first examine the autoencoder architectures, establishing the reconstruction floor, before evaluating the end-to-end surrogate models.

3.2 Autoencoder Reconstruction Floor

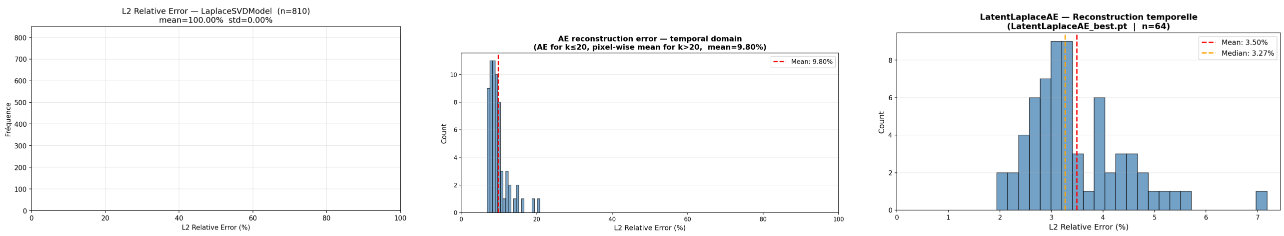
The autoencoders establish the best possible reconstruction accuracy achievable from the compressed latent representations, assuming perfect predictions from the surrogate.

Spatial Laplace AE (SLAE). The SLAE processes $K = 21$ uniform Bromwich frequencies ($k_{\max} = 20$) with a damping factor $\gamma = 0$. The architecture employs an encoder with 3 strided convolutional downsampling layers and a symmetric decoder with transposed convolutions. Both networks utilise FiLM conditioning with a sinusoidal positional encoding ($L = 8$) to process different frequencies with shared weights. The latent dimension is $d_z = 64$. The SLAE provides a robust non-linear spatial compression, achieving a median temporal reconstruction error of 9.17%, which sets the baseline floor.

Latent Laplace AE (LLAE). The LLAE processes single-channel physical frames using a time-conditioned convolutional encoder and decoder (conditioned on the normalised time step t/T via FiLM, $L = 8$), compressing each frame to $d_z = 64$. The temporal sequence of latent codes undergoes a differentiable Laplace transform over $K = 16$ learnable frequencies, initialised with $\gamma = 0.01$. The inverse Laplace transform is stabilised using Tikhonov regularisation ($\alpha_t = 7 \times 10^{-3}$, $\lambda = 3 \times 10^{-5}$).

Laplace SVD Baseline. As a linear baseline, a truncated SVD is applied independently to each Laplace frequency across the training set ($d_{\text{SVD}} = 32$). While computationally simple, the linear basis truncation error is substantial, yielding a widespread error distribution that fails to adequately capture the highly non-linear dynamics of the convection-diffusion-reaction process.

Figure 1 presents the reconstruction error histograms for the linear SVD baseline and both autoencoder architectures.



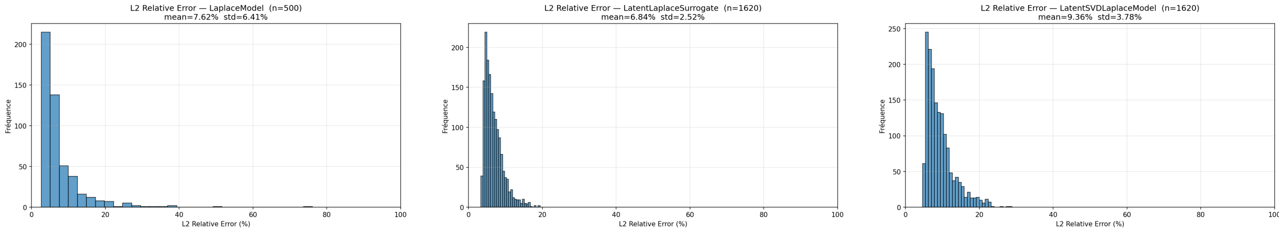
(a) Laplace SVD Baseline (b) SLAE Reconstruction Error (c) LLAE Reconstruction Error

Figure 1: Relative L^2 reconstruction error distributions on the test set. The linear SVD baseline struggles with the non-linear transients, whereas the autoencoders establish a much lower reconstruction floor for the surrogates.

3.3 Surrogate Prediction Performance

The surrogate models predict the latent representation directly from the physical parameters $\boldsymbol{\mu}$. For the latent Laplace models (LLAE and LSLAE variants), the surrogate architectures employ independent MLPs for each frequency, consisting of 4 layers with a trunk size of 256 and a head size of 128, mapping the 3-dimensional physical parameter vector to the complex frequency latents. For the LSLAE variant, the latent sequence is first projected offline using SVD to $d_{\text{SVD}} = 32$ modes to reduce the prediction dimensionality.

The overall prediction performances are evaluated for three configurations: the SLAE surrogate, the LLAE surrogate, and the LSLAE surrogate. Figure 2 illustrates the relative L^2 error distributions for these models.



(a) SLAE Surrogate (b) LLAE Surrogate (c) LSLAE Surrogate

Figure 2: Relative L^2 error distributions for the surrogate models over the test set. End-to-end training enables the autoencoder-based surrogates to bridge the gap toward their respective reconstruction floors.

The non-linear autoencoder-based models (SLAE, LLAE, and LSLAE surrogates) benefit significantly from the expressive capacity of the learned latent spaces and the end-to-end training strategy.

4 Comparison and Discussion

A comprehensive comparison of the methods is provided in Table 1.

Model	**Advantages**	**Drawbacks**	**Performance**
SLAE	Expressive non-linear spatial compression, frequency-conditioned (FiLM)	Memory intensive for computing all frequency frames	High
LLAE	Laplace transform applied on low-dimensional latents $z(t)$	Requires robust temporal consistency; computationally heavy (processes all N_t frames versus N_L), limiting network capacity under equal compute	High
LSLAE	Reduces latent dimensionality via SVD, simplifying the surrogate task; trained end-to-end	SVD truncation error in latent space; computationally heavy (processes all N_t frames)	High

Table 1: Comparison of surrogate strategies. End-to-end training is indispensable for the AE-based models.

4.1 Addressing Gibbs Oscillations: The Correction AE

The primary limitation of working in the Laplace domain is the necessary spectral truncation at a maximum frequency $k_{\max}$ to discard high-frequency noise. This truncation introduces Gibbs phenomena, manifesting as non-physical oscillations in the time domain $U_{\text{rec}}(t)$, particularly around sharp transients like explosion-like concentration onsets.

To mitigate these numerical artefacts, we introduce a lightweight **Correction Autoencoder (UNet residual corrector)**. Applied as a final post-processing patch, this network takes the inverted temporal fields $U_{\text{rec}}(t)$ and predicts a high-frequency residual correction $\delta_{\text{pred}}(t)$, smoothing the oscillations. The target residual is $\delta_{\text{true}}(t) = U(t) - U_{\text{rec}}(t)$. The corrector is trained by minimizing a normalized residual loss function that incorporates both MSE on the residuals and MSE on their spatial gradients to penalize spatial oscillations:

$$
\begin{aligned}
\mathcal{L}_{\text{Correction}} = & \left\| \frac{\delta_{\text{pred}}(t)}{\sigma} - \frac{\delta_{\text{true}}(t)}{\sigma} \right\|_2^2 \\
& + \lambda_{\text{grad}} \left(\left\| \frac{\nabla_x \delta_{\text{pred}}(t)}{\sigma} - \frac{\nabla_x \delta_{\text{true}}(t)}{\sigma} \right\|_2^2 + \left\| \frac{\nabla_y \delta_{\text{pred}}(t)}{\sigma} - \frac{\nabla_y \delta_{\text{true}}(t)}{\sigma} \right\|_2^2 \right)
\end{aligned} \tag{23}
$$

where σ is the standard deviation of $\delta_{\text{true}}(t)$ used as a normalization scaling factor, and λ_{grad} is a hyperparameter balancing the gradient penalty. This approach effectively smooths the oscillations and drastically reduces the median relative error at a marginal parameter cost. The final corrected error distribution is shown in Figure 3.

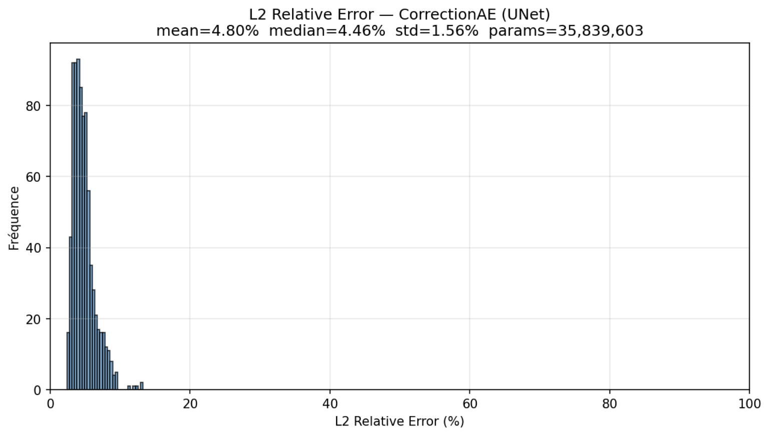


Figure 3: Relative L^2 error distribution for the UNet residual corrector. By smoothing out truncation artefacts, the corrector significantly improves the final temporal predictions.

Appendices

A Governing Equations

Our neural surrogate model is designed to emulate the transient dynamics of a reacting flow system. The physical system is governed by a set of coupled partial differential equations representing the conservation of mass, momentum, energy, and chemical species.

1. **Fluid Mechanics (Momentum):**

$$\frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U}\mathbf{U}) - \nabla \cdot \tau = -\nabla p + \rho \mathbf{g} \tag{24}$$

2. **Chemical Species Transport:**

$$\frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \mathbf{U} Y_i) - \nabla \cdot (\mu_{\text{eff}} \nabla Y_i) = \dot{\omega}_{i,\text{eff}} \tag{25}$$

under the constraint $\sum Y_i = 1$.

3. **Energy:**

$$\frac{\partial(\rho h)}{\partial t} + \nabla \cdot (\rho \mathbf{U} h) + \frac{\partial(\rho K)}{\partial t} + \nabla \cdot (\rho \mathbf{U} K) - \frac{\partial p}{\partial t} - \nabla \cdot (\alpha_{\text{eff}} \nabla h) = \dot{q}_{\text{chem}} \tag{26}$$

where $K = \frac{1}{2}|\mathbf{U}|^2$ is the kinetic energy.

4. **Combustion Model (PaSR):** The effective reaction rate is modulated by the Partially Stirred Reactor model:

$$\dot{\omega}_{i,\text{eff}} = \kappa \dot{\omega}_i(c^*, T^*), \quad \text{with} \quad \kappa = \frac{\tau_c}{\tau_c + \tau_{\text{mix}}} \tag{27}$$

where τ_c and τ_{mix} are the characteristic chemical and turbulent mixing times, respectively.

5. **Equation of State:** The ideal gas law closes the system:

$$p = \rho R T \quad \text{or} \quad \rho = \psi p \quad \text{with} \quad \psi = \frac{1}{RT} \tag{28}$$

B Architectural Details and Setup

All models were trained using the Adam optimizer with a learning rate of 5×10^{-4} for the autoencoders and 3×10^{-4} for the surrogates, over 300 epochs. The autoencoders used a batch size of 256 (SLAE) or 16 (LLAE). Training was performed on a workstation equipped with an Intel(R) Xeon(R) w3-2435 CPU (62GB RAM) and an NVIDIA GeForce RTX 3090 GPU (24GB VRAM). Training data consists of 1620 simulations; the test set contains 810 samples.

Table 2 and Table 3 provide a detailed summary of the network architectures and hyperparameters used for the various models discussed in this study.

Model	Encoder	Decoder	Conditioning head
SLAE	3 Conv2d (channels: 16, 32, 64, stride: 2) + LeakyReLU, 2 Linear layers	1 Linear, 3 ConvT2d (channels: 128, 64, 32, stride: 2) + ReLU, 3 Conv2d (channels: 32, 32, 1)	Positional Encoding + FiLM
LLAE	3 Conv2d (channels: 16, 32, 64, stride: 2) + LeakyReLU, 2 Linear layers	1 Linear, 3 ConvT2d (channels: 64, 32, 16, stride: 2) + ReLU, 3 Conv2d (channels: 16, 16, 1)	Positional Encoding + FiLM

Table 2: Network Architectures. Notation: Conv2d(in, out, k: kernel_size, s: stride, p: padding). All conv layers use LeakyReLU (encoder) or ReLU (decoder), and BatchNorm is applied in decoder ConvT2d layers.

Hyperparameter	SLAE	LLAE	LSLAE	Corrector
d_z	64	64	64	—
K	21	16	16	—
γ_{init}	0	0.01	0.01	—
d_{SVD}	—	—	32	—
β (latent ridge)	10^{-3}	10^{-2}	10^{-2}	—
α_t (Tikhonov smoothness)	—	7×10^{-3}	7×10^{-3}	—
λ (Tikhonov ridge)	—	3×10^{-5}	3×10^{-5}	—
α_{lat}	—	—	—	—
α_{spat}	—	—	—	—
λ_{grad}	—	—	—	10.0
Trunk / Head (MLP)	—	256 / 128	256 / 128	—
Base channels	—	—	—	16

Table 3: Hyperparameters for the Autoencoders, Surrogates, and Corrector models.

References

[1] Qianying Cao, Somdatta Goswami, and George Em Karniadakis. Lno: Laplace neural operator for solving differential equations. 2023.

[2] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling, 2025.

[3] Stefania Fresca, Luca Dede', and Andrea Manzoni. A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes. 2021.

[4] Matteo Giacomini and Pedro Díez. Surrogates for physics-based and data-driven modelling of parametric systems. 2026.

[5] Silke Glas, Antonia Mayerhofer, and Karsten Urban. Two ways to treat time in reduced basis methods. 2017.

[6] et al. Goeckner. Model order reduction in contour integral methods for parametric pdes. 2022.

[7] Nicola Guglielmi, María López-Fernández, and Giancarlo Nino. Numerical inverse laplace transform for convection-diffusion equations. 2020.

[8] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022.

[9] Jan S Hesthaven, Gianluigi Rozza, and Benjamin Stamm. *Certified reduced basis methods for parametrized partial differential equations*. Springer, 2015.

[10] Michael Urs Lars Kastor, Jan Rottmayer, Anna Hundertmark, and Nicolas Ralph Gauger. Parameter conditioned interpretable u-net surrogate model for data-driven predictions of convection-diffusion-reaction processes. 2026.

[11] Toni Lassila, Andrea Manzoni, Alfio Quarteroni, and Gianluigi Rozza. Model order reduction in fluid dynamics: Challenges and perspectives. 2014.

[12] Kookjin Lee and Kevin T Carlberg. Model reduction of dynamical systems on nonlinear manifolds using deep convolutional autoencoders. *Journal of Computational Physics*, 404:108973, 2020.

[13] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895.

[14] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.

[15] Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.

[16] Swaroop Mallick and Monika Mittal. Ai-based model order reduction techniques: A survey. 2025.

[17] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.

[18] Jad Mounayer, Sebastian Rodriguez, Chady Ghnatios, Charbel Farhat, and Francisco Chinesta. Rank reduction autoencoders. 2025.

[19] Eric Parish, Masayuki Yano, Irina Tezaur, and Traian Iliescu. Stabilization of cdr roms. 2025.

[20] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[21] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

[22] Ricardo Reyes. The frequency reduced-basis method: Reduced order models for time-dependent problems using the laplace transform. 2025.